

How to Run an Apache Spark Application

Sure! Let's talk about the concept of "spark-submit" in Apache Spark.

Understanding spark-submit: A Simple Explanation

In Apache Spark, "spark-submit" is like a magic key that allows you to start your Spark applications. Imagine you have a powerful car (your Spark application) that you want to drive on different roads (clusters). The "spark-submit" script is the key that helps you unlock the car and choose which road to take, whether it's a local road or a busy highway (cluster). By using this key, you can easily switch between different driving conditions just by changing one simple setting.

An Engaging Example

Think of "spark-submit" as a universal remote control for your TV. No matter what brand of TV you have, this remote can turn it on, change the channels, and adjust the volume. Similarly, "spark-submit" works with any type of Spark application, whether it's written in Python or Java, and it helps you connect to different cluster types without needing to learn a new way to operate each time. Just like how you can enjoy your favorite shows with one remote, you can run your Spark applications smoothly with "spark-submit."

What is 'spark-submit'

'spark-submit' script:

- Spark's Unified Interface for submitting applications
- Found in the 'bin/' directory
- Easily switches from local to cluster mode by changing a single argument
- Works the same way regardless of cluster manager type or application language

```
$ ./bin/spark-submit <config and options> <application files>
```

- Spark comes with a unified interface for submitting applications – it is the 'spark-submit' script found in the 'bin/' directory. 'spark-submit' can be used for all supported cluster types and accepts many configuration options for the application or cluster. "Unified interface" means you can switch from running Spark in local mode to cluster by changing a single argument. 'spark-submit' works the same way, no matter the application language. For example, a cluster can run Python and Java applications simultaneously by passing in the required files.

Using the 'spark-submit' Script

The 'spark-submit' script will:

1. Parse command line arguments/options
2. Read additional configuration specified in 'conf/spark-defaults.conf'
3. Connect to the cluster manager specified with the '--master' argument or run in local mode
4. Transfer application (JARs or Python files) and any additional files specified to be distributed and run in the cluster

- The 'spark-submit' script first parses any command line arguments or options. Then it reads any additional configuration specified in the 'spark-defaults' folder. Next, the '--master' argument tells the application how to connect to the cluster or run locally. Any application files (including JARs or Python files) must be specified so they can start the driver program and distribute files to run in the cluster.

Common 'spark-submit' Options

Option/Setting	Form	Mandatory ?
Tell Spark what cluster manager to connect with	'--master'	Yes
Specify the program entry point if using Java or Scala application	'--class <full-class-name>'	Yes
Set how the driver is deployed (Client or Cluster). Default is Client mode	'deploy-mode'	No
Set CPU core and memory usage in executors	'--executor-cores' and '--executor-memory'	No
See available options by cluster manager	'./bin/spark-submit -help'	N/A

- Some 'spark-submit' options are mandatory, such as specifying the master option to tell Spark which cluster manager to connect to. If the application is written in Java or Scala and packaged in a JAR, you must specify the full class name of the program entry point. Other options include driver deploy mode (run as a client or in the cluster) and executor resource settings (such as reserving memory or cores). Driver deploy mode defaults to client mode. Some options relate to specific cluster managers.

'spark-submit' Application Files

Final arguments depend on application language:

Java or Scala	Python
<code><application-jar-path> <application-args></code>	<code><application-py-script> <application-args></code>
Specifies the location of the JAR with your application and dependencies, followed by any arguments specific to the application	Specifies the application python script followed by any arguments specific to the application. Add files with '.py', '.egg' or '.zip' using the '--py-files' argument

- Additional Spark configuration can be specified on the command line using the '--conf' argument followed by key=value. For Java or Scala and Python, final arguments will be the path to the application JAR or Python script. Next are optional application-specific arguments. These will automatically be passed in. You can add Python files with '.py', '.egg', or '.zip' using the '--py-files' argument. These files will be distributed to the cluster and available in the Python environment.

'spark-submit' Examples

1. Launch Scala SparkPi using a jar, with master YARN. Estimate Pi with 1000 samples

```
# Launching Scala SparkPi to a YARN cluster

./bin/spark-submit \
  --class org.apache.spark.examples.SparkPi \
  --master YARN \
  /path/to/examples.jar \
  1000
```

2. Launch Python SparkPi to a Spark Standalone cluster. Estimate Pi with 1000 samples

```
# Launching Python SparkPi to a standalone cluster with master at 207.184.161.138

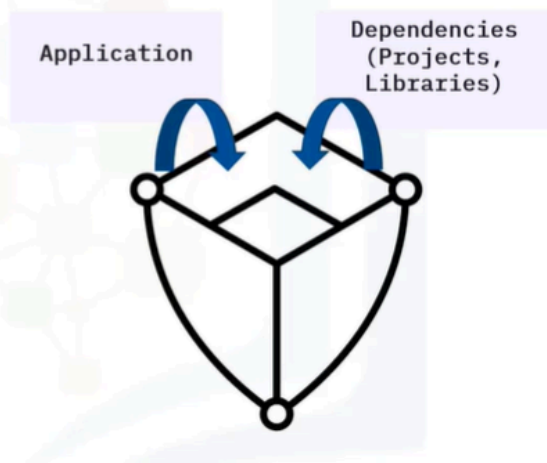
./bin/spark-submit \
  --master spark://207.184.161.138:7077 \
  examples/src/main/python/pi.py \
  1000
```

- Let's look at two 'spark-submit' examples. The first launches SparkPi, written in Scala and packaged in a JAR. YARN is the master. The application uses an additional argument (how many samples to take in the cluster to compute Pi). The second example launches a SparkPi Python version. The master is a Spark Standalone cluster and we simply pass the path to the Python script.

Application Dependencies

To manage Spark dependencies:

- Bundle projects or libraries with application so they are accessible to driver and executor processes
- For Java or Scala-based programs, create an uber-JAR with application and dependencies together so it is easy to distribute to the cluster



- To manage Spark dependencies, the application project or library must be accessible for the driver and cluster executor processes. For a Java or Scala-based application, it's best to create an uber-JAR. This is a single JAR file with all dependencies (including transitive ones) so the application is portable throughout the cluster.

Python Application Dependencies

To manage Python application dependencies, ensure:

- Cluster nodes can access the required dependencies with same version
- Same Python version is used
- You use the `--py-files` argument so that `.py`, `.zip` or `.egg` files can be distributed to the cluster

```
# Launching a PySpark application with dependency on "my_python_package"

$ ./bin/spark-submit <config and options> \
  --py-files my_python_package.zip \
  my_pyspark_application.py
```

- For Python applications, ensure that: (1) the cluster nodes access required dependencies with the same version AND (2) the same Python version is used. Solutions for Python dependencies may involve creating virtual environments so that applications can run in separate, isolated environments. Python dependencies can be included in the spark-submit script using the `--py-files` argument: this will accept `.py`, `.zip` or `.egg` files.

Spark Shell

Spark Shell:

- Simple way to learn Spark API
- Powerful tool to analyze data interactively
- Use in local mode or with a cluster, same options as 'spark-submit'
- Can initiate in Scala ('bin/spark-shell') and Python ('bin/pyspark')

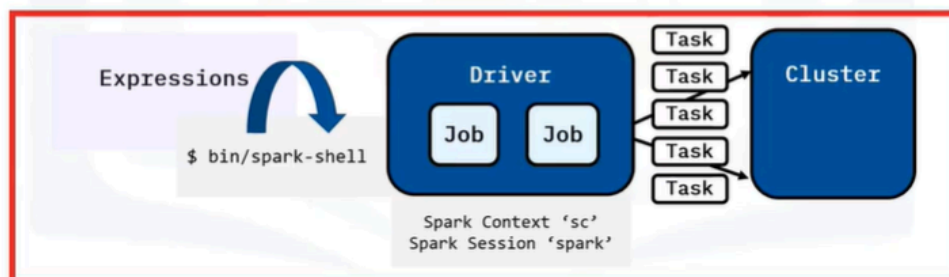


- Spark Shell is available for Scala and Python, giving you access to Spark APIs for working with data as Spark jobs. Spark Shell can be used in local or cluster mode, with all options available.

Spark Shell Environment

For both Scala and Python shells:

- SparkContext is automatically initialized and is available as 'sc'
- SparkSession is automatically available as 'spark'
- Expressions are entered into the shell and then evaluated in the driver to become jobs that are scheduled as tasks for the cluster



- When Spark Shell starts, the environment automatically initializes the SparkContext and SparkSession variables. This means you can start working with data immediately. Expressions are entered in the shell and evaluated in the driver. Entering an action on a shell DataFrame generates Spark jobs that are sent to the cluster to be scheduled as tasks.

Starting Spark Shell – Local Mode

Example: Start Scala Spark-Shell and run local mode by default

```
$ bin/spark-shell
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://192.168.1.33:4040
Spark context available as 'sc' (master = local[*], app id = local-1614631042181).
Spark session available as 'spark'.
Welcome to

 version 3.0.1

Using Scala version 2.12.10 (OpenJDK 64-Bit Server VM, Java 1.8.0_282)
Type in expressions to have them evaluated.
Type :help for more information.
scala>
```

Spark Shell Information Provided

- Spark's load log in the log4j-defaults.properties files
- Spark's Web UI address to view information about jobs running in the shell
- Spark Context and Spark Session variables
- Versions of important libraries in use, for example JDK and Scala

```
$ bin/spark-shell
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://192.168.1.33:4040
Spark context available as 'sc' (master = local[*], app id = local-1614631042181).
Spark session available as 'spark'.
Welcome to

 version 3.0.1

Using Scala version 2.12.10 (OpenJDK 64-Bit Server VM, Java 1.8.0 282)
Type in expressions to have them evaluated.
Type :help for more information.
scala>
```


Spark Shell Example – Run Code

```
scala> val df = spark.range(10)
df: org.apache.spark.sql.Dataset[Long] = [id: bigint]
```

```
scala> df.withColumn("mod", expr("id % 2")).show(4)
```

id	mod
0	0
1	1
2	0
3	1

1. Launch Scala Spark Shell
2. Create distributed DataFrame with column 'id' and 10 values (0-9)
3. Add a column that evaluates an SQL expression for modulo of 2 and show first four (4) results

- This example shows the Spark Scala shell starting up in local mode. The print-out includes: Spark's log in the log4j-defaults.properties file; Spark's Web UI address, displaying running jobs in the shell; Variable names for the initialized SparkContext and SparkSession, and Version information for important libraries like JDK and Scala. This example shows running code in the Scala shell. In the first line, a DataFrame is created as a distributed range of integers from 0 to 9. The second line transforms the DataFrame, adding a column calculating the modulo of 2. Next is an action that prints out the first four records of the resulting DataFrame.

Summary

In this video, you learned that:

- 'spark-submit' is a unified interface to submit the Spark application, no matter the cluster manager or application language
- Mandatory options include telling Spark which cluster manager to connect to; other options set driver deploy mode or executor resourcing
- To manage dependencies, application projects or libraries must be accessible for driver and executor processes, for example by creating a Java or Scala uber-JAR
- Spark Shell simplifies working with data by automatically initializing the SparkContext and SparkSession variables and providing Spark API access.